المراجعة الثالثة للمشروع: الاستكشاف وتجهيز البيانات للنمذجة

Exploratory Data Analysis & Leak-Free Preprocessing Pipeline

استخلاص الرؤى التحليلية (Business Insights)، وبناء خط معالجة خالٍ تماماً من تسريب البيانات (Data Leakage) لتحويل البيانات إلى نسق رقمي موحّد جاهز لتدريب النماذج.

Outputs: **train / validation / database_ready.csv**

Script: **split_with_figures.py**

Input: **engineered_data.csv** (1,000,000 rows)

MILESTONE 3 ARCHITECTURE & WORKFLOW

خارطة طريق التجهيز والانتقال من Milestone 2 إلى النمذجة

1. EDA & Correlations

استكشاف أنماط الاحتيال بصرياً وتحديد الارتباطات الإحصائية دون الخلط بين الارتباط والسببية.

Rubric: Insight Quality & Correlation

2. Split-First Pipeline

تقسيم البيانات طبقياً (Stratified Split) قبل أي خطوة Encoding أو Scaling لمنع Data Leakage.

Rubric: Leak-Free Preprocessing Order

3. Readiness Verification

التحقق الآلي من خلو المجموعات الثلاث من النصوص والقيم المفقودة وتطابق الأبعاد 100%.

Rubric: Final Dataset Readiness

تسلسل تنفيذ الأوامر في الكود (Execution Flow)

تم تطبيق القاعدة الذهبية: **Split First → Drop IDs → Fit/Align One-Hot on Train → Fit StandardScaler on Train Only.**

```
df = pd.read_csv("engineered_data.csv") # 1,000,000 rows, 28 cols
```

الرسوم الاستكشافية والرؤى التحليلية (EDA & Business Insights)

[PLACEHOLDER: ضع رسم الـ EDA الرئيسي هنا]

أدرج هنا رسم التوزيع أو المقارنة من دفتر EDA Notebook (مثل: معدل الاحتيال حسب `merchant_category` أو `hour_of_day` أو `is_international`).

1. الاستنتاج التجاري الأول (Insight #1 Placeholder)

[مثال لصياغة قوية حسب الـ Rubric: المعاملات الدولية التي تتجاوز X دولار تسجل احتمالية احتيال أعلى بـ X.Xx مرة مقارنة بالمعاملات المحلية].

2. الاستنتاج التجاري الثاني (Insight #2 Placeholder)

[مثال: ترتكز X% من حالات `is_fraud = 1` خلال ساعات محددة من `hour_of_day` مع تكرار `failed_attempts` قبل نجاح العملية].

معيار التقييم (Rubric Tip): ربط كل رسم بياني بقرار تشغيلي واضح بدلاً من الاكتفاء بوصف الشكل.

CORRELATION ANALYSIS • RUBRIC REQUIREMENT 2

تحليل الارتباط والتمييز بين الارتباط والسببية (Correlation ≠ Causation)

ماذا يثبت تحليل الارتباط؟ (What It Proves)

يقيس قوة واتجاه العلاقة الخطية المشتركة بين الميزات الرقمية والمتغير الهدف `is_fraud` لتحديد المتغيرات ذات القيمة التنبؤية.

ماذا لا يثبت؟ (Correlation ≠ Causation)

وجود ارتباط مرتفع بين ميزة (مثل `distance_from_home_km` أو `geo_velocity_proxy`) وبين `is_fraud` لا يعني أنها السبب المباشر، بل هي مؤشر سلوكي مرافق لنشاط اختراق الحساب.

[PLACEHOLDER: ضع صورة Correlation Heatmap هنا]

أدرج خريطة الارتباط من Seaborn واذكر أعلى 3 ميزات مرتبطة بـ `is_fraud` مع قيم معامل الارتباط (r) الخاصة بها.

الخلاصة التحليلية: نعتمد على الارتباط لاختيار الميزات (Feature Selection) وتجنب التعدد الخطي (Multicollinearity) دون ادعاء السببية المباشرة.

FEATURE ENGINEERING PORTFOLIO • RUBRIC REQUIREMENT 3

تبرير الميزات السلوكية المشتقة في `engineered_data.csv`

amount_deviation_score

يرصد القفزات المفاجئة في مبلغ المعاملة مقارنة بالسجل التاريخي السابق للعميل نفسه.

geo_velocity_proxy

يقسم المسافة على الفارق الزمني لكشف التنقل الجغرافي المستحيل فيزيائياً بين معاملتين.

failed_intl_interaction

يدمج المحاولات الفاشلة مع المعاملات الدولية لكشف هجمات تخمين البطاقات عبر الحدود.

balance_drain_ratio

يقيس نسبة مبلغ العملية إلى الرصيد المتاح لرصد محاولات تفريغ الحساب بالكامل.

```
# Prior-history features carried in engineered_data.csv
df['amount_deviation_score'] = ((amt - prior_mean) / (prior_std + 1e-3)).fillna(0)
df['geo_velocity_proxy'] = df['distance_from_home_km'] / (df['time_since_last_txn_hrs'] + 0.001)
df['failed_intl_interaction'] = df['failed_attempts'] * df['is_international']
df['balance_drain_ratio'] = df['transaction_amount'] / (df['account_balance'] + 0.001)
```

لماذا هذه الميزات خالية من التسريب؟ (Zero Leakage Reasoning)

تم حساب الإحصاءات التراكمية باستثناء الصف الحالي (`amt - cumsum()`) بعد الترتيب الزمني، بالإضافة إلى استخلاص `hour_of_day` مما يسمح بحذف التاريخ الخام لاحقاً.

STEP 1: DATA SPLITTING STRATEGY (SPLIT_WITH_FIGURES.PY)

الخطوة الأولى: التقسيم الطبقي المسبق لمنع تسريب البيانات

1. لماذا قسّمنا البيانات في بداية الكود؟

لتفادي الخطأ الأكثر شيوعاً في Milestone 3 وهو عمل Scaling أو Encoding قبل الـ Split، مما يسرب معلومات الاختبار للتدريب.

2. عزل مجموعة `database` (50% = 500k)

عزل نصف المليون صف كقاعدة بيانات إنتاجية خارجية لا يراها النموذج أبداً، وتقسيم النصف الآخر إلى 80% `train` و20% `validation`.

3. استخدام `stratify` و `random_state=42`

يضمن `stratify=df["is_fraud"]` تطابق نسبة الاحتيال في المجموعات الثلاث بسبب وجود Class Imbalance، مع ضمان Reproducibility.

```
# split_with_figures.py — Lines 9 to 27

df = pd.read_csv("engineered_data.csv")

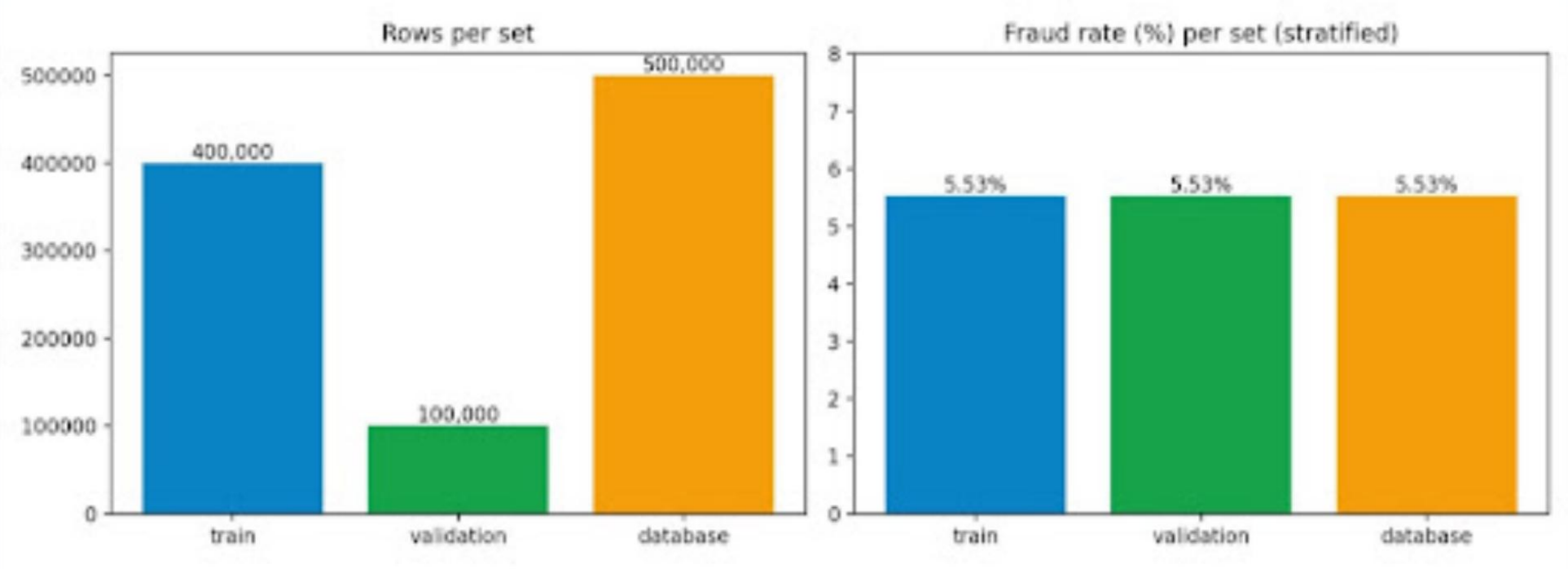
# Split off the "database" half (the model never sees it)
model_data, database = train_test_split(
    df, test_size=0.5, stratify=df["is_fraud"], random_state=42
)

# Split the model data: 80% train / 20% validation
train, val = train_test_split(
    model_data, test_size=0.2, stratify=model_data["is_fraud"], random_state=42
)

# Values saved for the "before" side of the figures
cols_before = train.shape[1]
text_cols_before = train.select_dtypes(exclude="number").shape[1]
amount_before = train["transaction_amount"].copy()
```

OUTPUT ARTIFACT 1 • FIG1_SPLIT_OVERVIEW.PNG

نتائج التقسيم الطبقي وتطابق نسبة الاحتيال عبر المجموعات



توزيع الصفوف (1,000,000 صف)

400,000 صف لـ `train`، و100,000 صف لـ `validation`، و500,000 صف لـ `database`.

ثبات معدل الاحتيال (5.53%)

نجح `Stratified Split` في تثبيت نسبة `is_fraud` عند **5.53%** بالضبط في كل مجموعة.

حذف المعرفات وتطبيق One-Hot Encoding المحكوم بـ Train

حذف المعرفات والتاريخ الخام (3 أعمدة)

تم إسقاط `customer_id` لمنع حفظ الهويات (Overfitting)، وحذف `transaction_date` و`transaction_time` لأن عمود `hour_of_day` يلتقط المعلومة الزمنية.

ترميز 5 أعمدة فئوية إلى 56 عموداً رقمياً

تحويل `country`, `city`, `merchant_category`, `payment_method`, `device_type` عبر `pd.get_dummies(..., dtype=int)` إلى أعمدة `0/1`.

أهمية `reindex(columns=train.columns).`

تُعرّف الأعمدة من `train` فقط؛ أي فئة جديدة في `val` أو `database` يتم تجاهلها وأي فئة غائبة تُملأ بـ `fill_value=0` لمنع التسريب.

split_with_figures.py — Lines 31 to 44

```
# 1) Drop columns not useful for the model: ID and raw date/time
drop_cols = ["customer_id", "transaction_date", "transaction_time"]
train = train.drop(columns=drop_cols)
val = val.drop(columns=drop_cols)
database = database.drop(columns=drop_cols)
cols_after_drop = train.shape[1]

# 2) Encode categorical columns (One-Hot) - defined from train only
cat_cols = ["country", "city", "merchant_category",
            "payment_method", "device_type"]
train = pd.get_dummies(train, columns=cat_cols, dtype=int)
val = pd.get_dummies(val, columns=cat_cols, dtype=int).reindex(
    columns=train.columns, fill_value=0)
database = pd.get_dummies(database, columns=cat_cols, dtype=int).reindex(
    columns=train.columns, fill_value=0)
```

STEP 4: LEAK-FREE FEATURE SCALING

توحيد المقاييس (StandardScaler) على الميزات المستمرة فقط

```
# split_with_figures.py — Lines 46 to 54

# 3) Scaling: fit on train only, then transform the others
target = "is_fraud"
features = [c for c in train.columns if c ≠ target]

# scale continuous columns only, not 0/1 columns
num_cols = [c for c in features if train[c].nunique() > 2]

scaler = StandardScaler()
train[num_cols] = scaler.fit_transform(train[num_cols])
val[num_cols] = scaler.transform(val[num_cols])
database[num_cols] = scaler.transform(database[num_cols])
```

استثناء الأعمدة الثنائية `nunique() > 2`

تم اختيار الأعمدة الرقمية المستمرة فقط لتطبيق الـ Scaling عليها، بينما تُركت أعمدة `0/1` وأعمدة الـ One-Hot الـ 56 دون تغيير للحفاظ على معناها المنطقي.

حساب المتوسط والانحراف من `train` فقط

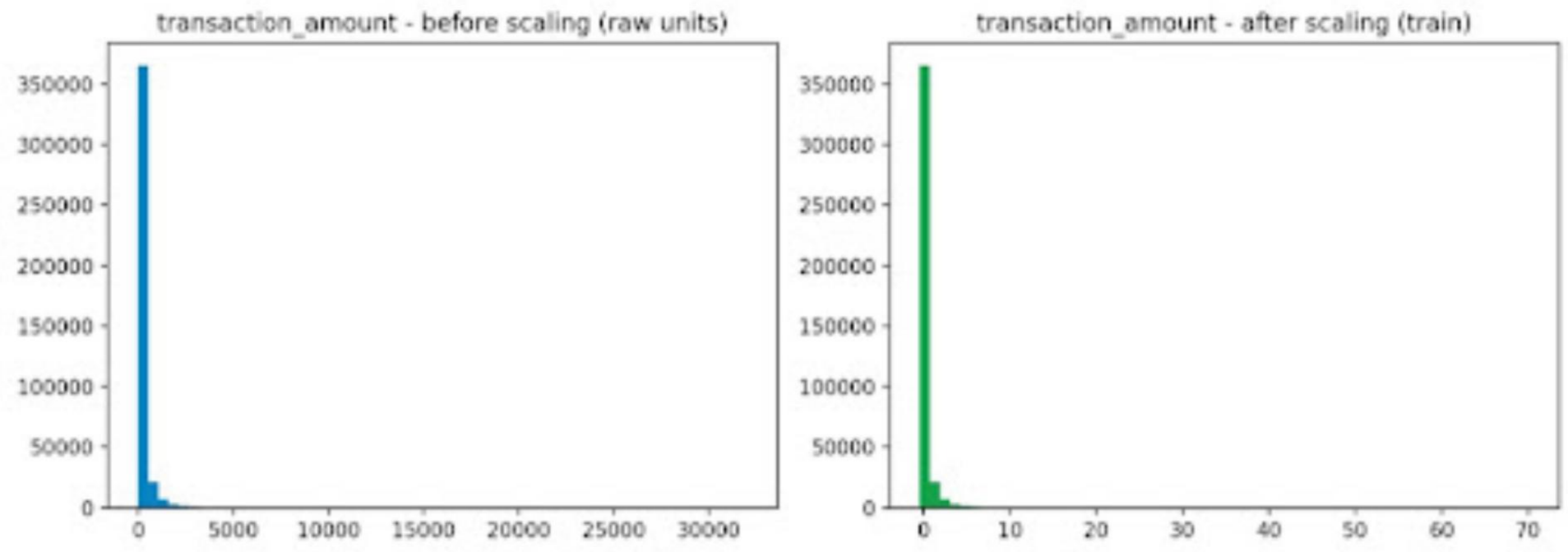
استخدمنا `scaler.fit_transform()` على `train` فقط، ثم طبقنا `scaler.transform()` على `val` و`database` باستخدام إحصائيات التدريب حصراً.

المعادلة الرياضية المطبقة: تحويل كل ميزة مستمرة لتصبح بمتوسط `Mean = 0` وانحراف معياري `Std = 1` داخل مجموعة التدريب.

$$z = \frac{x - \mu_{train}}{\sigma_{train}}$$

OUTPUT ARTIFACT 2 • FIG4_SCALING_EFFECT.PNG

أثر StandardScaler على توزيع ميزة `transaction_amount`



قبل التوحيد (Raw Units: 0 → 30,000+)

القيم الخام تتراوح بالآلاف، مما قد يجعل النماذج الحساسة للمسافة تنحاز للمبالغ الكبيرة.

بعد التوحيد (Standardized: Mean 0, Std 1)

تحول المحور الأفقي إلى درجات معيارية (Z-scores) مع الحفاظ التام على شكل التوزيع والـ Outliers الاحتيالية.

OUTPUT ARTIFACTS 3 & 4 • BEFORE/AFTER & READINESS VERIFICATION

مقارنة البيانات قبل وبعد المعالجة واجتياز فحوصات الجاهزية

Final Readiness Check

Check	Result
All columns are numeric	PASS
No missing values	PASS
Same columns in train / validation / database	PASS
No row overlap between sets	PASS
Scaler fitted on train only (train mean 0, std 1)	PASS

Before vs After Preprocessing

Metric	Before	After
Columns	28	76 (incl. target)
Text / ID columns	8	0
Categorical columns	5 text columns	56 one-hot numeric columns
Feature scale	Raw units	Standardized (mean 0, std 1)
Rows train / validation / database	1,000,000 (one file)	400,000 / 100,000 / 500,000

```
# 4) & 5) Readiness verification & saving model-ready files
no_overlap = (len(set(train.index) & set(val.index)) == 0
              and len(set(train.index) & set(database.index)) == 0
              and len(set(val.index) & set(database.index)) == 0)
train.to_csv("train_ready.csv", index=False)
val.to_csv("validation_ready.csv", index=False)
database.to_csv("database_ready.csv", index=False)
```

حفظ الملفات النهائية الجاهزة للنمذجة

بعد اجتياز جميع الفحوصات الخمسة بـ **PASS** (76 عموداً رقمياً بالكامل، 0 قيم مفقودة، 0 تداخل بين الصفوف)، يتم تصدير ملفات الـ CSV الثلاثة.

ZERO DATA LEAKAGE • 100% MODELING READY FOR MODULE 4

تم استكمال متطلبات Milestone 3 بنجاح

البيانات الآن مقسمة طبقياً، مرمّزة رقمياً بالكامل (76 عموداً)، وموحّدة القياس دون أي تسريب للمعلومات، وجاهزة للبدء الفوري بتدريب نماذج تعلم الآلة.

train_ready.csv

400,000 صف × 76 عموداً

validation_ready.csv

100,000 صف × 76 عموداً

database_ready.csv

500,000 صف × 76 عموداً

Thank You — Ready for Q&A and Live Defense